

공공입찰 RFP 질의응답 RAG 시스템 구축 및 성능 개선 보고서

코드잇 AI 스프린트 중급 프로젝트 (개인) · 이진호

작업 기간: 2026년 6월 22일 ~ 7월 8일

요약

이 프로젝트는 B2G 입찰지원 스타트업 '입찰메이트'의 사내 도구라는 가정 아래, 수십 페이지에 달하는 공공입찰 제안요청서(RFP) 100건에서 필요한 정보를 근거와 함께 찾아 답하는 RAG(검색 증강 생성) 시스템을 만든 기록이다. 클라우드 서버 없이 개인 PC와 OpenAI API만으로 구현했다.

진행하면서 가장 신경 쓴 부분은 "시스템이 돌아가느냐"가 아니라 "성능을 어떻게 측정하고, 실험으로 얼마나 개선했으며, 그 숫자를 어디까지 믿을 수 있느냐"였다. 검색 쪽에서는 발주기관 자동 필터와 대화맥락 기반 쿼리 재작성을 붙여 정답 문서를 훨씬 잘 찾게 만들었고, 답변 근거성은 알았던 CSV 추출본 대신 원본 hwp/pdf를 통째로 파싱하는 방식으로 바뀌 끌어올렸다.

다만 솔직하게 말하면 이 시스템은 완벽하지 않다. 튜닝셋에서는 종합 점수(EDD)가 95점대까지 올랐지만, 한 번도 보지 않은 검증셋에서는 81점(v2), 89점(v3)으로 떨어졌다. 튜닝셋 숫자가 그대로 일반화되지 않는다는 걸 스스로 확인한 셈이다. 대표 답변 8개를 사람 눈으로 다시 채점했을 때는 평균 3.6점(5점 만점)에 그쳤다. 그래서 이 보고서는 성능이 오른 지점과 함께, 어떤 숫자는 왜 조심해서 봐야 하는지도 같이 남긴다.

1. 프로젝트 개요

공공입찰 시장에서는 하루에도 수백 건의 RFP가 나라장터 같은 곳에 올라온다. 문제는 한 건이 수십 페이지라 컨설턴트가 일일이 읽기 어렵다는 점이다. 결국 사업의 주요 요구조건, 대상 기관, 예산, 제출 방식 같은 핵심만 빠르게 뽑아낼 수 있는 도구가 필요하고, 이 프로젝트는 그 도구를 RAG로 만드는 것을 목표로 삼았다.

개인 프로젝트라 팀에 제공되는 GCP 서버를 쓸 수 없었다. 그래서 로컬 환경에서 OpenAI API를 호출하는 방식(가이드의 시나리오 B)으로 갔다. 허용된 모델인 gpt-5-mini와 text-embedding-3-small 범위에서 구현했고, 로딩부터 청킹, 임베딩, 검색, 생성, 평가까지 전 구간을 다뤘다.

처음에는 "RAG 말고 파인튜닝을 해야 하나" 하는 고민도 있었다. 하지만 두 방법은 목적은 같아도 성격이 다르다. 파인튜닝은 모델 가중치를 다시 학습시켜 지식을 각인시키는 방식인데, 사실을 정확히 외우는 데 약하고 오히려 환각이 늘며 GPU와 라벨 데이터가 필요하다. 반면 RAG는 질문마다 관련 문서를 검색해 프롬프트에 넣어주기 때문에, 문서가 바뀌면 재색인만 하면 되고 근거를 인용할 수 있어 환각이 적다. 100건의 문서에서 사실을 정확히 찾아 답하는 이번 과제에는 RAG가 정석이라고 판단했고, 과제 요구사항 자체도 RAG였다.

2. 데이터와 전처리

제공받은 데이터는 실제 RFP 100건(hwp 96개, pdf 4개)과 이를 정리한 `data_list.csv`였다. CSV에는 공고번호, 사업명, 사업금액, 발주기관, 마감일, 사업요약 같은 메타데이터가 들어 있었다. 먼저 CSV의 파일명과 실제 파일이 정확히 맞는지 확인했는데 100건 모두 일치했고, 고유한 발주기관은 87곳이었다. 원본 RFP는 비밀유지계약 대상이라 저장소나 보고서에는 원문을 실지 않고 가공된 결과만 사용했다.

데이터를 열어보다가 CSV에 텍스트 컬럼이 있는 걸 발견했다. 본문이 어느 정도 미리 추출돼 있어서, 초기 베이스라인을 빠르게 만드는 데 요긴했다. 그런데 문서당 평균 글자 수가 3,800자 정도로, 원문의 극히 일부에 불과했다. 나중에 실험에서 확인했지만, 이 얇은 텍스트 때문에 검색은 잘 되어도 세부 요구사항이 답변에 담기지 않아 근거성이 떨어지는 문제가 생겼다.

그래서 중간에 원본 hwp/pdf를 직접 통째로 파싱하는 방식으로 전환했다. 그 결과 문서당 평균 글자 수가 약 73,000자로, CSV 대비 19배까지 늘었고 100건 전부 파싱에 성공했다.

원본 파싱은 생각보다 까다로웠다. 처음 만든 HWP 파서는 본문에 정체불명의 제어문자 잡음이 섞여 나왔는데, HWP 본문 규칙상 인라인 컨트롤 문자가 8워드를 차지한다는 점을 반영해 걸러냈다. 그다음에는 임베딩을 보내다가 `UnicodeEncodeError`가 터졌다. 원인을 파고드니 원문에 기본다국어평면(BMP) 밖 문자가 있었고, 파서가 UTF-16 서로게이트 쌍을 두 조각으로 쪼개면서 인코딩이 깨진 것이었다. 상위·하위 서로게이트를 하나로 결합하고, API에 보내기 직전 한 번 더 정제하는 안전장치를 두어 해결했다. 바이너리 포맷 파싱에서는 유니코드 엡지케이스를 반드시 방어해야 한다는 걸 몸으로 배웠다.

3. 시스템 구조

전체 흐름은 다음과 같다. 사용자 질문이 들어오면, 후속질문인 경우 대화 맥락을 반영해 독립적인 질문으로 다시 쓴 뒤, 임베딩해서 Chroma에서 관련 청크를 검색한다. 이때 발주기관 자동 필터와 MMR을 적용해 근거를 추린다. 추려진 근거를 컨텍스트로 넣어 gpt-5-mini가 근거 기반 답변을 생성하고, 답변에는 어느 문서에서 나온 내용인지 표시한다.

구성요소와 선택 이유를 정리하면 아래와 같다.

구성	선택	이유
임베딩	text-embedding-3-small	허용 모델이고 저비용에 성능이 충분
벡터 DB	Chroma	로컬에 영속 저장 가능, 메타데이터 필터링 지원
청킹	글자수 800 / 중첩 150	문단 경계를 존중하며 근거 단위 확보
검색	코사인 유사도 + MMR + 자동 기관필터	관련성과 다양성을 함께 잡고 발주기관으로 범위를 좁힘
생성	gpt-5-mini	허용 모델, 근거 기반 프롬프트로 환각 억제

검색에서 특히 공을 들인 부분이 두 가지다. 하나는 발주기관 자동 필터다. 질문 속에서 기관명을 감지해 발주 기관 메타데이터로 검색 범위를 좁히는데, 사용자가 기관명을 정확히 쓰지 않아도(예: '원자력연구소') 잡아내도록 했고, 여러 기관을 비교하는 질문에서는 관련 기관을 모두 포함하도록 만들었다. 다른 하나는 대화맥락 쿼리 재작성이다. "그중에 예약 기능은?" 같은 후속질문은 기관명이 생략돼 그대로 검색하면 엉뚱한 문서로 새기 때문에, 직전 대화를 반영해 혼자서도 검색 가능한 질문으로 바꾼 뒤 검색하게 했다.

4. 평가 설계

성능을 감이 아니라 숫자로 보고 싶어서 평가셋과 지표를 직접 설계했다.

평가셋은 단일 문서 추출, 여러 문서 종합, 후속 맥락 유지, 기관 비교, 부정확한 기관명, 정확한 수치 확인, 없는 정보에 대한 거절(abstention), 함정 질문 같은 유형으로 구성했다. 각 질문에는 정답 발주기관을 라벨로 붙여 검색이 제대로 됐는지 객관적으로 채점할 수 있게 했다. 평가셋은 프로젝트가 진행되면서 초기 10문항에서 25문항으로, 다시 오타나 줄임말, 억지 질문까지 섞은 12문항으로 계속 진화시켰다. 그리고 과적합을 막기 위해 25문항을 튜닝용 18개와 검증용 7개로 나누고, 검증용은 최종 확인 전까지 튜닝에 쓰지 않았다.

지표는 검색과 답변, 환각 방지, 효율로 나눠 봤다. 검색은 정답 기관이 검색 결과에 포함되는 비율(coverage), 모든 정답 기관을 다 잡았는지(hit - all), 첫 정답의 순위(MRR)를 봤다. 답변은 근거에 충실한지(groundedness)와 질문에 실제로 답하는지(relevance)를 LLM 채점으로 봤다. 환각 방지는 없는 정보에 "모른다"고 답하는지(abstention accuracy)와, 반대로 정상 답변을 거절로 잘못 분류하지 않는지를 봤다. 마지막으로 응답 지연을 췄다.

이 지표들을 한눈에 비교하려고 EDD라는 종합 점수를 정의했다. coverage 20%, hit - all 10%, MRR 15%, groundedness 20%, relevance 20%, abstention 10%, latency 5%로 가중합하고, 없는 답을 거절로 오분류하거나 답변이 비면 패널티로 감점했다. 다만 EDD는 어디까지나 비교를 돕는 보조 점수이고, 진짜 판단은 원지표로 했다.

채점을 얼마나 믿을 수 있느냐도 중요한 문제였다. 답변한 모델이 자기 답을 다시 채점하면 점수가 부풀 수 있어서, 답변 모델과 채점 모델을 분리했다. 또 채점기가 검색된 근거를 앞부분만 보고 뒷부분을 놓치는 버그가 있어서, 근거 전체를 균등하게 보도록 고쳤다. 여기에 더해 일부러 틀린 답(있는 결제 근거를 없다고 하거나, 낙찰업체를 날조하는 답 등)을 섞은 함정 검증팩을 만들어, 채점기가 그런 오답을 제대로 떨어뜨리는지 확인하는 블라인드 검증을 설계했다. 이 검증의 실제 실행은 시간 관계상 향후 과제로 남겼다.

5. 실험과 개선 과정

개선은 파라미터를 무작정 바꾸는 식이 아니라, 어떤 질문이 왜 틀렸는지 진단하고 그 실패에 맞는 처방을 하나씩 적용한 뒤 다시 측정하고 비판적으로 되짚는 순서로 진행했다.

검색 기법 비교 (CSV 모드, 10문항)

먼저 CSV 텍스트로 여러 검색 설정을 비교했다.

실험	coverage	MRR	groundedness	relevance	latency
베이스라인(유사도 top5)	0.562	0.542	1.43	1.43	10.9s
MMR top5	0.5	0.525	2.0	2.0	11.3s
MMR top8	0.625	0.543	2.0	2.25	9.1s
MMR + 자동필터	0.625	0.75	1.63	1.88	7.7s
MMR + LLM 리랭크	0.562	0.542	1.67	1.67	19.7s

자동 기관필터가 확실히 좋았다. MRR이 0.54에서 0.75로 뛰었고 지연시간도 가장 낮았다. 질문에 있는 기관명으로 검색 범위를 좁히니 정답 문서가 1순위로 올라온 덕이다. 반대로 LLM 리랭크는 검색 지표는 그대로면서 지연만 두 배가 됐다. 후보가 이미 괜찮을 때는 리랭킹이 별 이득 없이 비용만 든다는 걸 확인했고, 이런 "안 되는 실험"도 근거로 남겼다.

이 실험에서 더 큰 문제 두 가지가 드러났다. 하나는 답변 품질이 전반적으로 낮다는 것이었는데(groundedness 1.4~2.2), 원인은 CSV 텍스트가 알아서였다. 그래서 앞서 말한 원본 파싱 전환을 결정했다. 다른 하나는 후속질문이 검색에 계속 실패한다는 것이었고, 이것 해결하려고 쿼리 재작성을 도입했다.

원본 전환 후 조건 비교 (12조건)

원본으로 재인덱싱하니 청크가 616개에서 11,842개로 늘었다. 이 위에서 top_k, mmr_lambda, fetch_k, 자동필터, 쿼리재작성을 조합해 어블레이션했다. 가장 좋은 조건은 MMR top8에 mmr_lambda 0.5, fetch_k 20, 자동필터와 쿼리재작성을 모두 켜 설정이었고, EDD 96.57에 coverage와 MRR이 1.0, groundedness 4.667, relevance 5.0이 나왔다. top5도 근소하게 따라왔지만 문맥 여유를 위해 top8을 기본값으로 정했다.

여기서 배운 점은 자동필터와 쿼리재작성이 서로 다른 실패를 메운다는 것이었다. 필터를 끄면 기관명이 명시된 질문이 깨지고, 재작성을 끄면 후속질문이 깨진다. 그래서 둘 중 하나가 아니라 둘 다 필요했다. 또 mmr_lambda를 0.7로 올리거나 fetch_k를 40으로 키우면 검색 지표는 유지되지만 근거성이 오히려 떨어졌다. 후보를 무작정 많이 넣는 게 능사가 아니었다.

지표 포화와 측정계 재설계

이쯤 되니 coverage와 MRR이 1.0, relevance가 5.0으로 계속 반복됐다. 처음엔 시스템이 완벽해진 줄 알았지만, 다시 생각해보니 이건 "평가셋이 너무 쉽다"는 신호였다. 포화된 지표 위에서 파라미터만 바꾸면 실제 개선이 아니라 노이즈를 쫓게 된다. 그래서 멀티홉, 비슷하지만 없는 사업, 애매한 기관명, 정확한 수치 확인 같은 더 어려운 25문항으로 다시 켜더니 EDD가 79.90으로, groundedness가 3.125로 푹 떨어지면서 진짜 약점이 보였다. 답변이 근거에 없는 버전명이나 세부 기능을 지어 넣는 문제였다.

세 가지 보정과 그 정직한 해석

여기서 세 가지를 고쳤다. 첫째, 채점기가 근거를 앞부분만 보던 문제를 고쳐 전체를 균등하게 보게 하니 groundedness가 회복되며 EDD가 79.90에서 84.79로 올랐다. 다만 이건 답변이 좋아진 효과와 채점 방식이 바뀐 효과가 섞인 것이라 확대해석하지 않으려 했다. 둘째, 비교 질문에서 누락되는 기관을 채워 넣고 별칭(재단법인, 행정구역 접두어 제거 등)으로도 매칭하게 보강했다. 셋째, 정상적인 "확인할 수 없습니다" 답변을 거절로 올바르게 인식하도록 판정을 고쳤다. 세 보정을 합치니 튜닝셋 EDD가 95.55까지 올랐다.

검증셋으로 드러난 과적합

여기까지만 보면 성능이 크게 오른 것 같지만, 진짜 시험은 검증셋이었다. 튜닝셋 95.55짜리 설정을 한 번도 보지 않은 검증 7문항에 적용하니 EDD가 81.55로 떨어졌다. 튜닝셋 숫자가 그대로 일반화되지 않는다는 게 숫자로 드러난 순간이었다. 어떤 문항이 틀렸는지(비교질문의 기관 누락, 없는 정보 거절 실패) 진단해서 고친 뒤 같은 검증셋을 다시 재니 96.80이 나왔지만, 이건 엄밀히 말해 일반화 성능이 아니라 "같은 셋의 실패를 보고 고친 뒤 재측정한 값"이다. 그래서 이 숫자에는 반드시 꼬리표를 달아야 한다고 봤다.

이후 완전히 새로운 12문항(v3)으로 다시 검증했다. 원점수는 EDD 89.69였고, 채점 판정만 보정하면 96.36까지 올랐다. 속도를 위해 top5로 낮추는 후보도 시험했지만, 안양 호계체육관 예약시스템 질문처럼 결제 · PG 세부근거가 필요한 문항에서 품질이 떨어져서 전역 기본값으로는 채택하지 않았다.

사람 기준 품질 리뷰

자동 지표가 아무리 높아도 사람이 읽었을 때 좋은 답변인지는 별개다. 그래서 대표 답변 8개를 골라 답변 자체 품질, 근거 안전성, 보고서 활용성 세 관점으로 직접 채점했다. 결과는 품질 평균 3.62점, 근거 안전성 평균 3.47점(5점 만점)이었다. 자동 EDD가 90점을 넘던 것과 대조적으로 꽤 냉정한 숫자였고, "맞긴 한데 장황하다"거나 "근거는 맞지만 다른 사업의 조항이 섞일 위험이 있다" 같은 실제 숙제를 드러내 줬다.

6. 결과

최종적으로 정한 기본 설정은 원본 파싱 데이터에 MMR top8, mmr_lambda 0.5, fetch_k 20, 자동필터와 쿼리재작성을 켜고 LLM 리랭크는 끈 구성이다. 평균 응답 지연은 대략 19~25초다.

성능은 국면별로 다음과 같이 정리할 수 있다.

평가 국면	EDD	성격
v2 튜닝셋(18문항)	95.55	튜닝 성능. 과적합 가능성 있음
v2 검증셋(7문항) 첫 측정	81.55	가장 정직한 일반화 근거
v2 검증셋(7문항) 보정 후 재측정	96.80	같은 셋 재측정. 일반화 아님
v3 새 검증셋(12문항) 원점수	89.69	새 질문 첫 측정
사람 기준 리뷰(8개)	3.62 / 5	실제 체감 품질

정리하면 검색은 분명히 개선됐고(정답 문서를 1순위로 회수), 원본 파싱 덕에 근거성도 올랐다. 하지만 처음 보는 질문에서는 EDD가 80점대이고 사람 체감 품질은 3.6 수준이다. "완벽한 시스템"이라기보다는 "약점이 어디 있는지 분명히 아는 시스템"에 가깝다.

7. 주요 의사결정 요약

결정	결과	근거
GCP 미사용, API 기반(시나리오 B)	채택	개인 규모에 적합, 완성도 우선
벡터 DB로 Chroma	채택	로컬 영속 + 메타데이터 필터
원본 hwp/pdf 전체 파싱	채택	CSV 텍스트가 알아 근거성 저하
발주기관 자동 필터	채택	MRR 0.54 → 0.75
대화맥락 쿼리 재작성	채택	후속질문 검색 실패 해결
LLM 리랭크	폐기	검색 개선 없이 지연만 2배
전역 top5(속도 우선)	보류	일부 문항에서 품질 하락

8. 한계와 개선 사항

첫째, 채점기가 후한지 아직 완전히 검증하지 못했다. groundedness가 오른 데에는 채점 방식을 바꾼 효과가 섞여 있는데, 함정 검증팩까지는 준비했지만 블라인드 실행은 못 했다. 이 검증 전까지 groundedness의 절대값은 조건부로 봐야 한다.

둘째, 검증셋 규모가 7~12문항으로 작다. 한두 문항만 뒤집혀도 EDD가 크게 출렁이기 때문에, 96.80 같은 소수점 숫자를 정밀한 성능처럼 다루면 안 된다.

셋째, 답변에 다른 사업의 조항이 섞이는 근거 범위 오염이 일부 남아 있다(사람 리뷰 3.47이 이를 가리킨다). bullet마다 출처 범위를 엄격히 제한하는 인용 가드가 다음 과제다.

넷째, 응답 지연이 19~25초로 느리다. 원본 컨텍스트가 길기 때문인데, 컨텍스트 압축이나 쉬운 질문에만 top5를 쓰는 적응형 방식으로 줄일 여지가 있다.

다섯째, 관찰된 실패 패턴에 맞춰 넣은 규칙성 처리들이 있어서, 완전히 새로운 질문에는 일반화가 제한적일 수 있다. 실제 운영 시스템이라면 이런 부분을 학습 기반이나 더 일반적인 처리로 대체하는 편이 낫다.

9. 결론

이 프로젝트로 RFP 100건에 대한 RAG 질의응답 시스템을 완성했고, 검색과 답변 근거성을 실험으로 개선했다. 하지만 개인적으로 가장 의미 있었던 성과는 높은 점수 자체가 아니라, 성능을 어떻게 측정했고 어떤 개선이 진짜였으며 어떤 숫자는 왜 못 믿는지를 규명하고 기록했다는 점이다. 지표가 포화된 걸 알아채고 더 어려운 평가로 다시 설계했고, 튜닝셋 95.55가 검증셋 81.55로 떨어지는 과적합을 스스로 잡아냈으며, 사람 기준 3.62라는 냉정한 값까지 남겼다. 이 과정이 오히려 시스템의 신뢰성을 뒷받침한다고 생각한다.

다음 단계로는 채점기 블라인드 검증, 근거 범위 오염 방지, 응답 속도와 간결성 개선을 남겨 둔다.

부록 A. 진행 중 해결한 주요 기술 이슈

1. 공고번호 결측으로 문서 ID가 충돌한 문제 — 행번호와 접미사를 붙여 유일하게 만들
2. LLM 채점이 계속 빈 값으로 나온 문제 — 추론형 모델이 토큰을 다 써서였고, 토큰을 늘리고 JSON 응답을 강제해 해결
3. HWP 서로게이트 인코딩 오류 — 서로게이트 쌍 결합과 인코딩 정제로 해결
4. Chroma 한 번에 저장할 수 있는 개수 상한(5,461) — 5,000개씩 나눠 저장
5. 자동필터가 비교질문에서 한 기관만 남기거나 엉뚱한 기관을 잡던 문제 — 정밀 부분일치와 다중 포함으로 수정
6. 인덱스가 비었는데 groundedness가 높게 나오던 착시 — 빈 인덱스 가드 추가
7. 답변이 통째로 비는 문제 — 생성 토큰을 늘리고 재시도 로직 추가
8. 정상 답변을 거절로 오분류하던 문제 — 판정 기준을 좁히고 페널티로 반영

부록 B. 재현 방법


```
pip install -r requirements.txt
# .env 에 OPENAI_API_KEY 설정
python -m scripts.build_index raw # 원본 파싱 인덱스 구축 (약 11,842 청크)
python -m scripts.ask # 대화형 질의
python -m scripts.run_experiments # 설정 비교 실험
```

원본 RFP 데이터와 API 키는 저장소에 포함하지 않는다(.gitignore 처리).